

KVCompressionTune: A Study on Workload-Aware KV-Cache Compression Configuration for an Open-Source LLM Serving Engine

Avinoam Nukrai Jad Mahajne

August 2026

1 Introduction

Large language model (LLM) serving is increasingly constrained by the memory consumed by the key-value (KV) cache, whose size grows linearly with sequence length and the number of concurrent sequences. For long-context and high-concurrency deployments, the KV cache can dominate GPU memory and limit the number of requests that can be served concurrently. KV-cache quantization addresses this bottleneck by storing cached keys and values at lower precision, trading some model quality for increased cache capacity [1, 2].

We build on **vLLM**, a widely used open-source LLM serving engine [1], and specifically on **TurboQuant**, its built-in KV-cache quantization mechanism, merged upstream in April 2026. TurboQuant uses Hadamard rotation followed by Lloyd–Max scalar quantization for its 3- and 4-bit key modes and uniform quantization for values, while the **k8v4** preset stores keys in FP8 [2, 3]. vLLM 0.21.0 exposes four named TurboQuant presets (**k8v4**, **4bit_nc**, **k3v4_nc**, **3bit_nc**), with documented storage-compression ratios ranging from approximately $2.6\times$ to $4.9\times$ before accounting for layer protection [3]. For dense models, the implementation additionally protects the first two and last two layers by keeping them at full precision; this lowers the whole-model compression ratio. Our tuner accounts for the protected layers using the analytical payload model described in Section 3.

This representation choice is distinct from vLLM’s block-based KV-cache memory management through PagedAttention and its optional prefix-caching mechanisms. PagedAttention manages how KV-cache blocks are allocated and shared; TurboQuant determines how the keys and values stored in those blocks are represented [1]. Our extension therefore operates on the representation/configuration axis and is complementary to cache allocation, reuse, and eviction mechanisms.

Despite TurboQuant’s integration, practitioners still face a deployment-specific configuration problem: which preset should be used for a given model, hardware platform, and workload? The answer is non-obvious for three reasons that we substantiate empirically in this report. First, the offline PPL degradation used to enforce our quality budget is not normally available as an online quality signal for free-form production requests, so an overly aggressive configuration can violate the deployment’s quality constraint without an automatic warning from vLLM. Second, preset feasibility can be hardware-dependent: in our stock vLLM 0.21.0 tests, **k8v4** failed to initialize on an RTX 3090 (Ampere, SM 8.6) while running successfully on an RTX 4090 (Ada, SM 8.9), whereas the three **_nc** presets ran on both. Third, the same preset can have dramatically different effects across models: **3bit_nc** increases PPL by 54.6% on Qwen3-1.7B but only 0.69% on Mistral-7B-v0.3 in our screening experiments.

The broader observation that quantization sensitivity is non-uniform across layers is well established. HAWQ demonstrated layer-dependent sensitivity for weight quantization using Hessian information [4]; KVQuant introduced per-layer sensitivity-weighted non-uniform datatypes for KV-cache quantization [5]; and KVTuner and KVMix explicitly search or allocate layer-wise mixed precision [6, 7]. RateQuant instead formulates mixed-precision allocation across attention heads using rate-distortion theory [8]. These methods target finer-grained quantization policies. Our project asks a different systems question: given the TurboQuant configurations already exposed by stock vLLM, can we select a configuration

according to deployment-specific objectives such as chat decode latency, RAG prefill latency and cache capacity, or batch throughput, while enforcing a quality constraint?

This report therefore asks a narrower, practical question: given the presets vLLM already ships, does the best choice depend on workload and model, and can that choice be automated? We answer both empirically. Reading the vLLM source also showed that TurboQuant’s component parameters (key precision, value precision, and norm correction) are represented separately internally even though the CLI exposes only four named presets, and that user-supplied protected layers are added to the default boundary floor rather than replacing it. Both observations shape our experimental grid and suggest small, well-scoped opportunities for an upstream vLLM contribution.

2 Extension Design

The baseline system—stock vLLM with TurboQuant—exposes four fixed compression presets but provides no built-in mechanism for choosing among them according to the model, hardware, workload, and quality requirement. Our extension adds a policy-selection layer on top without modifying vLLM’s source: given a *profiled* model, target GPU, and workload, it recommends the preset and protection budget that maximizes a workload-specific utility function subject to a hard quality constraint. The recommendation is applied at engine-launch time; live per-request reconfiguration inside a running vLLM engine is outside the scope of this project. Every component below is implemented and tested, and none requires a vLLM fork or a custom kernel.

No Workload-Aware Configuration Selection: A vLLM user selects one static `--kv-cache-dtype` preset when launching an engine, and that configuration remains fixed for the lifetime of that engine. vLLM does not connect this choice to a deployment-specific workload objective or an explicit measured quality budget. We address this with a decision engine that chooses a configuration for each profiled (model, GPU, workload) deployment context using measurements collected by our benchmark and tuner.

TurboQuant-Specific Layer Sensitivity Was Uncharacterized: Prior work already establishes that quantization sensitivity can vary across layers [4, 5, 6, 7]; what was unclear for our setting was how sensitivity is distributed under TurboQuant on the models we study, and whether vLLM’s fixed boundary-protection rule protects the layers that matter most. We built a **layer sensitivity profiler** with two backends: a fast PyTorch reimplement of TurboQuant’s quantization math that hooks HuggingFace Transformers to rank layers, and a slower vLLM ground-truth backend that quantizes exactly one real layer at a time through vLLM’s engine. The latter is used to validate whether the fast backend measures the same phenomenon as the deployed implementation.

No Standardized, Reproducible Benchmarking: Comparing configurations fairly requires identical, repeatable workloads. We built a **benchmark harness** that launches an unmodified vLLM engine with a given configuration, replays a frozen, seeded workload trace, and records latency, throughput, device-level VRAM, quality, and retrieval metrics. Traces are frozen to JSON so every configuration cell sees byte-identical prompts, and completed cells are checkpointed by a content hash so an interrupted cluster job can resume instead of restarting.

No Automatic Preset or Protection Selection: Using the screening data, we built a **workload-aware tuner** that searches the (preset $\times$ protection budget) space with Optuna’s TPE sampler, warm-started from the screening grid. Each workload profile supplies its own multiplicative utility function (Table 1), combining relative latency/throughput improvement with the analytical memory-compression ratio. A configuration whose PPL degradation exceeds the workload’s threshold receives zero utility regardless of its speed or memory benefit.

No Auditable, Hardware-Aware Recommendation: Finally, a **config advisor** takes a profiled (model, GPU, workload) triple and returns recommended vLLM launch flags together with evidence such as measured Δ PPL and utility. It also applies the feasibility rules encoded from our hardware tests and can fall back to the next-best feasible configuration when the top preset is unavailable on the target GPU.

3 Experimental Setup

Workload Profiles and Quality Budgets: We evaluate three deployment profiles because they value different parts of serving performance. *Chat* uses short prompts followed by relatively long generation, so

the user mainly feels the speed of token generation after the first token. We therefore make chat primarily TPOT-sensitive and give it the strictest quality budget. *RAG* uses long retrieved context followed by a short answer, so processing the prompt and producing the first token are especially important; *RAG* therefore emphasizes TTFT and cache capacity. *Batch* represents offline bulk processing, where the main goal is to process as many output tokens as possible per second, so it emphasizes throughput.

Quality is measured relative to the FP16 baseline using

$$\Delta\text{PPL}_{\text{rel}} = \frac{\text{PPL}_{\text{cfg}} - \text{PPL}_{\text{FP16}}}{\text{PPL}_{\text{FP16}}}.$$

A configuration is considered viable only if this relative PPL increase stays below the workload’s threshold: 0.5% for chat, 1.0% for *RAG*, and 2.0% for batch. If a configuration exceeds the threshold, the tuner assigns it utility 0, regardless of its speed or compression benefit. All prompts are synthetically generated and seeded for determinism; screening/tuning use one seed while final validation replays a disjoint held-out seed that was never used during tuning.

Latency and Throughput Metrics: The three performance metrics used by the utility functions measure different stages of inference:

- **Time to First Token (TTFT)** is the time from the request’s arrival until the first generated output token becomes available. It includes scheduling and prompt/prefill processing before the first token is produced. Lower TTFT is better. This is especially important for *RAG*, where the prompt is long and the generated answer is relatively short.
- **Time Per Output Token (TPOT)** measures generation speed *after* the first output token. For a request with end-to-end latency T_{e2e} , TTFT T_{first} , and $N_{\text{out}} > 1$ generated tokens, our harness computes

$$\text{TPOT} = \frac{T_{\text{e2e}} - T_{\text{first}}}{N_{\text{out}} - 1}.$$

Lower TPOT means subsequent tokens appear faster. This is the main latency metric for chat because chat requests generate many tokens after the first one.

- **Throughput (TP)** measures how much generation work the system completes per unit time. For a workload trace,

$$\text{TP} = \frac{\text{total number of generated output tokens}}{\text{wall-clock time for the workload}} \quad \text{tokens/s.}$$

Higher throughput is better. This is the main performance metric for batch processing.

We additionally record device-level VRAM with a background NVML sampling thread, WikiText-2 perplexity under chunked prefill, and needle-in-haystack retrieval accuracy as a long-context sanity check.

Normalized Performance Terms: The tuner does not combine raw milliseconds and tokens/s directly. Instead, each candidate configuration is compared with the FP16 baseline, producing dimensionless improvement factors:

$$S_{\text{TPOT}} = \frac{\text{TPOT}_{\text{FP16}}}{\text{TPOT}_{\text{cfg}}}, \quad S_{\text{TTFT}} = \frac{\text{TTFT}_{\text{FP16}}}{\text{TTFT}_{\text{cfg}}}, \quad S_{\text{TP}} = \frac{\text{TP}_{\text{cfg}}}{\text{TP}_{\text{FP16}}}.$$

The ratios are intentionally oriented so that **larger is always better**: a value of 1 means the candidate matches FP16, a value greater than 1 means an improvement, and a value below 1 means a regression. For example, $S_{\text{TPOT}} = 1.10$ means that the candidate generates subsequent tokens about 10% faster than the baseline in this normalized score.

Compression Model: Because vLLM pre-allocates a KV-cache pool, peak allocated VRAM does not directly reveal how many more tokens fit after quantization. The tuner therefore uses an analytical cache-capacity ratio, R_{mem} . For a model with L layers and s protected full-precision layers,

$$R_{\text{mem}} = \frac{4L}{4s + (L - s)b_{\text{preset}}},$$

where FP16 keys plus values are modeled as 4 bytes per dimension and b_{preset} is 1.5 for `k8v4`, 1.0 for `4bit_nc`, 0.875 for `k3v4_nc`, and 0.75 for `3bit_nc`. Thus, $R_{\text{mem}} = 1$ is the FP16 baseline, while $R_{\text{mem}} = 3$

means that the analytical KV payload is three times smaller, or equivalently that roughly three times as much KV data can fit in the same payload budget. This is the exact analytical model used by `src/tuner.py`; it is not a direct measurement of the packed physical KV-cache footprint and does not include all implementation metadata.

Workload Utility Functions: The tuner combines the normalized performance term that matters most for each workload with R_{mem} using a weighted geometric mean:

Table 1: Utility function and PPL constraint per workload profile (`configs/profiles.json`).

Workload	Utility	PPL constraint
Chat	$U_{\text{chat}} = S_{\text{TPOT}}^{0.7} R_{\text{mem}}^{0.3}$	$\leq 0.5\%$
RAG	$U_{\text{rag}} = R_{\text{mem}}^{0.5} S_{\text{TTFT}}^{0.5}$	$\leq 1.0\%$
Batch	$U_{\text{batch}} = S_{\text{TP}}^{0.8} R_{\text{mem}}^{0.2}$	$\leq 2.0\%$

The exponents express the workload priorities. Chat gives 70% of the utility weight to faster generation (TPOT) and 30% to cache compression. RAG weights cache compression and TTFT equally. Batch gives 80% of the weight to throughput and 20% to compression. Because every normalized term equals 1 for FP16, the FP16 baseline has utility $U = 1$. A viable candidate with $U > 1$ is preferred to FP16 under that workload’s priorities, while $U < 1$ is worse overall. The quality constraint is applied *before* this comparison: if the candidate exceeds the allowed ΔPPL , its utility is set to 0 even if it is faster or more compressed.

Hardware and Models: Primary hardware is an RTX 4090 (Ada, SM 8.9); an RTX 3090 (Ampere, SM 8.6) is used specifically as a cross-generation feasibility check. The primary model is Qwen3-4B (36 layers); Qwen3-1.7B (28 layers), Llama-3.1-8B, and Mistral-7B-v0.3 are used for the generalization study. We also attempted Gemma-2-2B; under our vLLM 0.21.0 setup, all TurboQuant presets failed after the unrelated initial length configuration issue was corrected (discussed in Step 5).

Software Environment: All reported vLLM experiments use stock vLLM 0.21.0. Optuna is used for the tuner’s search, and the current test suite contains 60 collected pytest tests covering the profiler, harness, workload/profile logic, tuner, advisor, and model-topology helpers. The repository’s `requirements.txt` pins vLLM 0.21.0 for reproduction.

Protocol: The study runs in five cumulative phases: layer-sensitivity profiling; a screening grid over presets and protection sets; workload-aware auto-tuning with a protection-budget ablation; held-out statistical validation; and cross-model/cross-architecture generalization.

4 Results & Discussion

Step 1: Baseline Characterization — Layer Sensitivity Profiling

Before tuning anything, we profiled per-layer sensitivity to 3-bit key quantization (with 4-bit values) in isolation, to examine whether vLLM’s fixed first/last-two-layer floor protects the layers that matter most in Qwen3-4B.

Key Observations (Figure 1):

- Layer 0 is by far the most sensitive measured layer: its ΔPPL is +4.59, compared with +0.24 for the next-worst measured layer (about $19\times$ larger).
- Layer 1—protected by the floor alongside layer 0—is nearly insensitive (+0.01), while layers 30–33 outside the floor show measurable sensitivity (+0.07 to +0.15).
- Local simulated quantization error correlates with downstream sensitivity (Spearman $\rho = 0.40$, $p = 0.016$), but imperfectly: layer 5 has high local error yet effectively zero end-to-end impact. This mismatch becomes important in Step 4.

Step 2: From Sensitivity Profiling to Preset Screening

With per-layer sensitivity mapped, we ran a 34-cell screening grid on Qwen3-4B / RTX 4090: the FP16 baseline (2 reps, no protection-axis variation) plus all four TurboQuant presets crossed with four

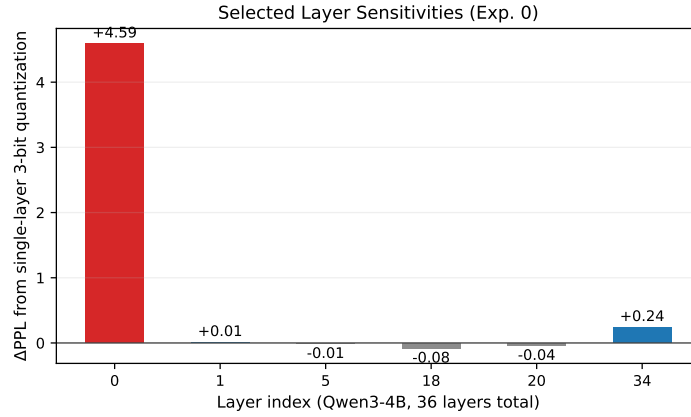


Figure 1: Per-layer Δ PPL for layers with a directly measured value (Qwen3-4B, single-layer 3-bit isolation). Red: layer 0 (most sensitive). Blue: layers the default floor protects. Grey: unprotected.

protection sets and two reps each ($2 + 4 \times 4 \times 2 = 34$). The purpose was to determine whether preset choice or protection choice dominates the quality/performance trade-off.

Key Observations (Table 2, Figure 2):

- The mildest preset (**k8v4**) matches the FP16 baseline within the observed screening noise; the most aggressive preset (**3bit_nc**) costs about +3.4% PPL, a 0.36-PPL spread.
- The best protection scheme recovers at most about 0.09 PPL for a fixed preset. On this screening metric, preset choice therefore produces roughly four times the PPL range of protection choice.
- Decode overhead (TPOT) increases with quantization aggressiveness: about +9% for **k8v4**, rising to roughly +16–19% for **3bit_nc**.
- Needle-in-haystack retrieval remains near 1.00 across almost all configurations; the main outlier is **k8v4** with floor-only protection (0.95).

Table 2: Preset screening summary, Qwen3-4B / RTX 4090, floor protection. Compression is the analytical, floor-adjusted R_{mem} used by the tuner; it is not a direct packed-memory measurement.

Preset	PPL	Δ PPL	Analytical R_{mem}
FP16 baseline	10.35	—	1.00×
k8v4	10.34	−0.10%	2.25×
4bit_nc	10.45	+0.93%	3.00×
k3v4_nc	10.66	+3.01%	3.27×
3bit_nc	10.70	+3.38%	3.60×

These findings expose several trade-offs that motivate workload-specific selection:

- **Prefill and decode can favor different configurations.** Aggressive quantization clearly raises TPOT because decode pays quantization/dequantization overhead. In early screening we also observed large TTFT improvements under some compressed configurations, but TTFT at two repetitions was too noisy for a strong standalone claim; the held-out workload utilities in Step 4 therefore use five repeated measurements.
- **Peak allocated VRAM does not shrink in the way a naive measurement suggests.** vLLM v1 pre-allocates a KV-cache pool according to its memory budget, so compression primarily increases the number of tokens/requests that fit in that pool rather than proportionally lowering peak device allocation. We therefore use R_{mem} as a capacity proxy rather than raw peak-VRAM reduction.
- **Hardware feasibility precedes optimality.** In our vLLM 0.21.0 smoke tests, **k8v4** failed to initialize on the RTX 3090 while all three **_nc** presets ran; the same **k8v4** preset ran on the RTX 4090.

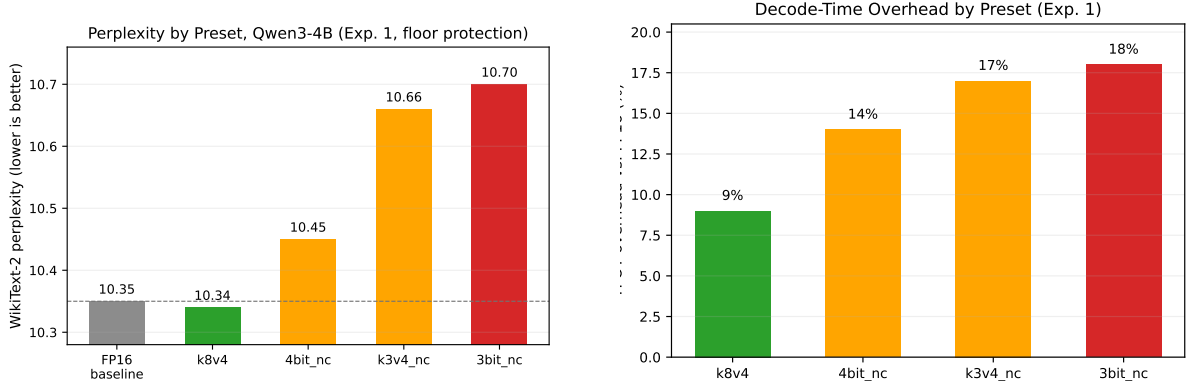


Figure 2: (a) WikiText-2 perplexity and (b) decode-time overhead, by preset (floor protection).

The advisor therefore treats feasibility as a hardware-dependent filter before utility optimization.

Step 3: From Screening to Auto-Tuning and a Protection-Budget Ablation

Preset choice clearly dominates the screening PPL range, but does extra protection help once a preset is fixed? We ran the Optuna tuner over 36 configurations (20 unique preset/budget configurations carried over from screening plus 16 refinement cells) and separately examined the protection budget for `k3v4_nc`.

Key Observations (Figure 3):

- The tuner selects different launch-time configurations for the three workload profiles: chat $\rightarrow$ `k8v4`; RAG $\rightarrow$ `4bit_nc` with layer 5 additionally protected; batch $\rightarrow$ `4bit_nc` with floor-only protection.
- For `k3v4_nc`, adding layer 33 accounts for about 90% of the total PPL recovery from the extra protection explored around that operating point, moving PPL from 10.662 to 10.570. A third extra protected layer improves PPL by a further 0.009.
- The best measured budget is 3 extra layers (PPL 10.561). Budgets 4–8 were also measured and show no additional improvement or slight regression, so the benefit saturates rather than increasing monotonically with protection budget.
- Even at budget 3, `k3v4_nc` misses the batch PPL threshold (10.554) by 0.007 PPL; protection closes most of the gap without crossing the constraint.

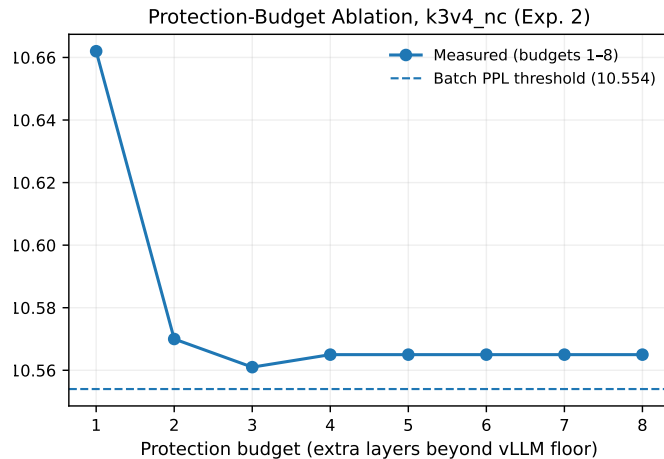


Figure 3: Protection-budget ablation for `k3v4_nc`. All budgets 1–8 were measured; improvement saturates after budget 3 and higher budgets are flat-to-regressing.

Step 4: From Auto-Tuning to Held-Out Statistical Validation

To avoid selecting and evaluating on the same traces, we re-evaluated the configurations chosen during tuning on disjoint held-out traces that were not used during screening or tuning (5 repetitions). For latency/throughput comparisons we report paired t -tests and Wilcoxon signed-rank tests with Holm–Bonferroni correction across 30 comparisons. PPL is deterministic under our teacher-forced evaluation, so repeated PPL values do not provide an informative sampling distribution; we therefore report PPL differences directly rather than interpreting their repeated-run p -values.

Key Observations (Table 3, Figure 4):

- Utility, normalized so the FP16 baseline scores 1.0, improves by +19.4% for chat, +57.6% for RAG, and +14.8% for batch for the pre-selected tuner configurations.
- `3bit_nc` fails every workload’s quality threshold on held-out data ($\Delta\text{PPL} = +3.43\%$), so the most aggressive preset is not viable for any of the three Qwen3-4B profiles under our quality budgets.
- Latency metrics have low but non-zero run-to-run variance and support the paired performance comparisons; PPL is identical across repetitions for a fixed configuration and trace set.

Table 3: Held-out validation of the configurations selected during tuning, Qwen3-4B / RTX 4090, $n = 5$ reps. Utility gain is relative to FP16. Compression is analytical R_{mem} .

Workload	Preset	R_{mem}	ΔPPL	Utility gain
Chat	k8v4	$2.25\times$	-0.02%	+19.4%
RAG	4bit_nc+L5	$2.82\times$	+0.91%	+57.6%
Batch	4bit_nc	$3.00\times$	+0.98%	+14.8%

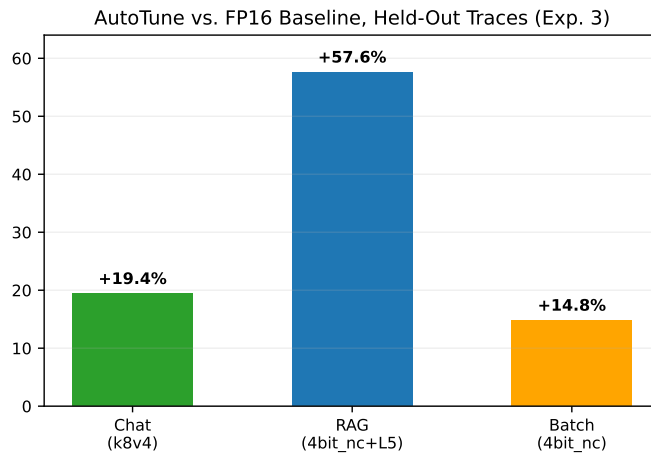


Figure 4: Utility gain of the pre-selected configuration over the FP16 baseline on held-out traces.

Two trade-offs are especially important:

- **Workload-aware selection is useful even though one conservative preset can be feasible everywhere on this GPU.** Applying `k8v4` to all three Qwen3-4B workloads satisfies the quality thresholds, but leaves about 8.4% RAG utility and 2.7% batch utility unclaimed relative to the workload-specific choices. Conversely, applying `4bit_nc` globally improves utility for RAG/batch but violates chat’s 0.5% PPL threshold. Thus, the result is not that no global preset is feasible; it is that no single tested preset is simultaneously the best choice for all three objectives.
- **Statistics-guided layer protection is a negative result on held-out data.** The positional extra-protection choice (layer 2) produces slightly lower PPL than the statistics-guided layer-5 choice (10.436 vs. 10.442), while the utility difference is not statistically significant ($p = 0.33$). More importantly, floor-only `4bit_nc` has higher RAG utility than `4bit_nc+L5` on held-out data because both pass the PPL constraint and the added protection reduces R_{mem} . This shows that

the profiler is useful for characterization, but its local error ranking does not reliably translate into a better end-to-end protection policy.

Step 5: From Single-Model Validation to Cross-Model Generalization

Finally, we asked whether the model dependence observed on Qwen3-4B extends across model size and architecture family. Qwen3-1.7B, Llama-3.1-8B, and Mistral-7B-v0.3 each received a two-repetition screening-phase sweep using the same workload harness. These are real committed measurements, but unlike Qwen3-4B they have not yet received the independent five-repetition held-out validation protocol.

Key Observations (Table 4, Figure 5):

- Qwen3-1.7B is far more fragile than Qwen3-4B: `3bit_nc` costs +54.6% PPL versus +3.4% on Qwen3-4B (about a $16\times$ difference), while `4bit_nc` costs about six times more relative PPL.
- `k8v4` improves measured PPL on Qwen3-1.7B (−2.03%). This could reflect a regularization-like effect or evaluation variation, but we did not run additional experiments to establish a mechanism.
- Llama-3.1-8B and especially Mistral-7B-v0.3 are more robust than the Qwen models under the screened presets. Mistral tolerates `3bit_nc` for RAG at only +0.69% PPL and obtains the largest screening-phase utility gain in the study (+73.8%).
- For Qwen3-1.7B batch, no TurboQuant candidate beats the FP16 baseline in utility; the best TurboQuant candidate, `k8v4`, scores 0.989, so the correct recommendation under this profile is to keep the FP16 baseline.

Table 4: Best tested TurboQuant configuration and utility gain relative to FP16 per (model, workload). Qwen3-4B uses held-out validation ($n = 5$); other models are screening-phase ($n = 2$). The Qwen3-1.7B batch row is shown for comparison even though FP16 is the actual optimum.

Model	WL	Preset	Δ PPL	Util. gain
Qwen3-4B	Chat	<code>k8v4</code>	−0.02%	+19.4%
	RAG	<code>4bit_nc+L5</code>	+0.91%	+57.6%
	Batch	<code>4bit_nc</code>	+0.98%	+14.8%
Qwen3-1.7B	Chat	<code>k8v4</code>	−2.03%	+24.9%
	RAG	<code>k8v4</code>	−2.03%	+10.0%
	Batch	<code>k8v4[†]</code>	−2.03%	−1.1%
Llama-3.1-8B	Chat	<code>4bit_nc</code>	+0.39%	+30.9%
	RAG	<code>4bit_nc</code>	+0.39%	+57.7%
	Batch	<code>3bit_nc</code>	+1.78%	+20.4%
Mistral-7B-v0.3	Chat	<code>4bit_nc</code>	+0.05%	+30.4%
	RAG	<code>3bit_nc</code>	+0.69%	+73.8%
	Batch	<code>3bit_nc</code>	+0.69%	+18.7%

[†]Below baseline utility (1.0); FP16 is the recommended choice for this workload.

Two measurement caveats are important when interpreting the cross-model results:

- **Llama’s needle-retrieval metric is unreliable in our harness.** It is 0.0 under every tested configuration, including FP16, indicating a model-family prompt/chat-template mismatch rather than a quantization effect. We therefore exclude this metric from claims about Llama quantization quality.
- **Gemma-2-2B is incompatible with TurboQuant in our tested vLLM 0.21.0 setup.** The first run failed because of an unrelated `max_model_len` configuration error. After correcting it, all `turboquant.*` presets still failed with `NotImplementedError` in `unify_kv_cache_spec_page_size`, consistent with Gemma-2’s alternating local/global attention producing KV-cache specifications that this TurboQuant path could not unify. We report this as a version/setup-specific compatibility finding rather than claiming universal incompatibility with all future vLLM versions.

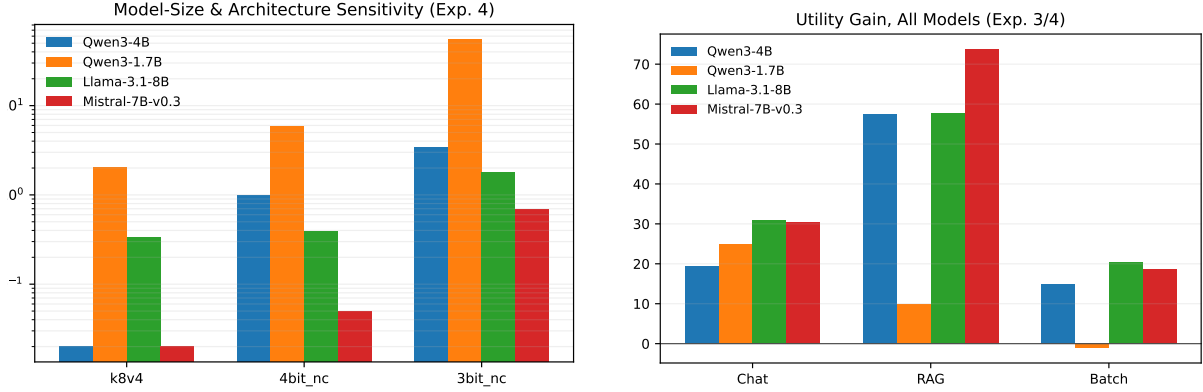


Figure 5: (a) $|\Delta\text{PPL}|$ relative to FP16 on a log scale. (b) Utility gain over FP16 across the four successfully evaluated models and three workload profiles.

5 Conclusion

This project shows that the best KV-cache compression preset is workload-dependent and model-dependent, and that this selection can be automated without modifying vLLM’s source. A configuration that is acceptable for one workload can violate another workload’s quality budget, and the same aggressive preset that has little effect on Mistral-7B-v0.3 can severely degrade Qwen3-1.7B. On Qwen3-4B / RTX 4090, `k8v4` is a conservative globally feasible choice, but it is not utility-optimal for RAG or batch; more aggressive global choices such as `4bit_nc` improve those workloads but fail the chat quality constraint. The practical value therefore comes from choosing a configuration for the deployment objective rather than assuming one preset is universally best.

We also report an important negative result: statistics-guided per-layer protection does not outperform simpler positional/floor protection on held-out utility. In the screening grid, preset choice produced about four times the PPL range of protection choice, and on held-out RAG data the added layer-5 protection slightly improves PPL but lowers utility because it sacrifices analytical cache capacity after the quality threshold is already satisfied. The main contribution is therefore workload- and model-aware preset selection, with layer sensitivity serving primarily as characterization rather than as the dominant optimization lever.

Key contributions include a dual-backend layer-sensitivity profiler, a reproducible benchmark harness with frozen seeded traces and content-addressed checkpoints, a workload-aware Optuna tuner, and a hardware-aware config advisor with an auditable recommendation. The current repository contains 60 collected tests. Across the primary study, cross-model sweeps, layer profiling, and the Gemma compatibility attempt, the project executes more than 150 experimental configurations/runs; full five-repetition held-out validation is currently limited to Qwen3-4B.

Future Work. Several directions remain open. *Per-request configuration* within a single running engine would require changes to vLLM’s allocator and attention/quantization path rather than launch-time flags. *More accurate capacity modeling* could replace the tuner’s payload-level R_{mem} proxy with measured usable KV-block capacity or a packed-storage model that includes metadata. *Per-model quantizer calibration* could test whether the stock scalar codebooks remain optimal for different model-specific rotated KV distributions. *Extending the full $n = 5$ held-out validation protocol* to Qwen3-1.7B, Llama-3.1-8B, and Mistral-7B-v0.3 would give the model-dependence claim the same statistical weight as the primary Qwen3-4B result. Finally, two small upstream candidates remain: exposing more of TurboQuant’s internal configuration space through the public interface, and making the dense-model boundary-protection width configurable rather than fixed at two layers per side.

All code, raw experiment cells, analysis scripts, and tests are available at <https://github.com/AvinoamNukrai/KVCompressionTune>. The committed `requirements.txt` pins vLLM 0.21.0, the version used for the reported vLLM experiments.

References

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023.
- [2] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. *ICLR*, 2026.
- [3] vLLM Project. TurboQuant implementation documentation, vLLM 0.21.0. https://docs.vllm.ai/en/v0.21.0/api/vllm/model_executor/layers/quantization/turboquant/.
- [4] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ: Hessian AWARE Quantization of Neural Networks with Mixed-Precision. *ICCV*, 2019.
- [5] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. arXiv:2401.18079, 2024.
- [6] X. Li, Z. Xing, Y. Li, L. Qu, H.-L. Zhen, Y. Yao, W. Liu, S. J. Pan, and M. Yuan. KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference. *ICML*, 2025.
- [7] F. Li, S. Liu, W. Wu, S. Nie, and J. Wang. KVMix: Gradient-Based Layer Importance-Aware Mixed-Precision Quantization for KV Cache. *AAAI*, 2026.
- [8] F. Zuo, Z. Zhou, H. Cong, X. Xi, and H. F. Leung. RateQuant: Optimal Mixed-Precision KV Cache Quantization via Rate-Distortion Theory. arXiv:2605.06675, 2026.